# Trees

# Scope



Trees:

- Trees as data structures
- Tree terminology
- Tree implementations
- Tree traversals
- Expression trees

# Trees

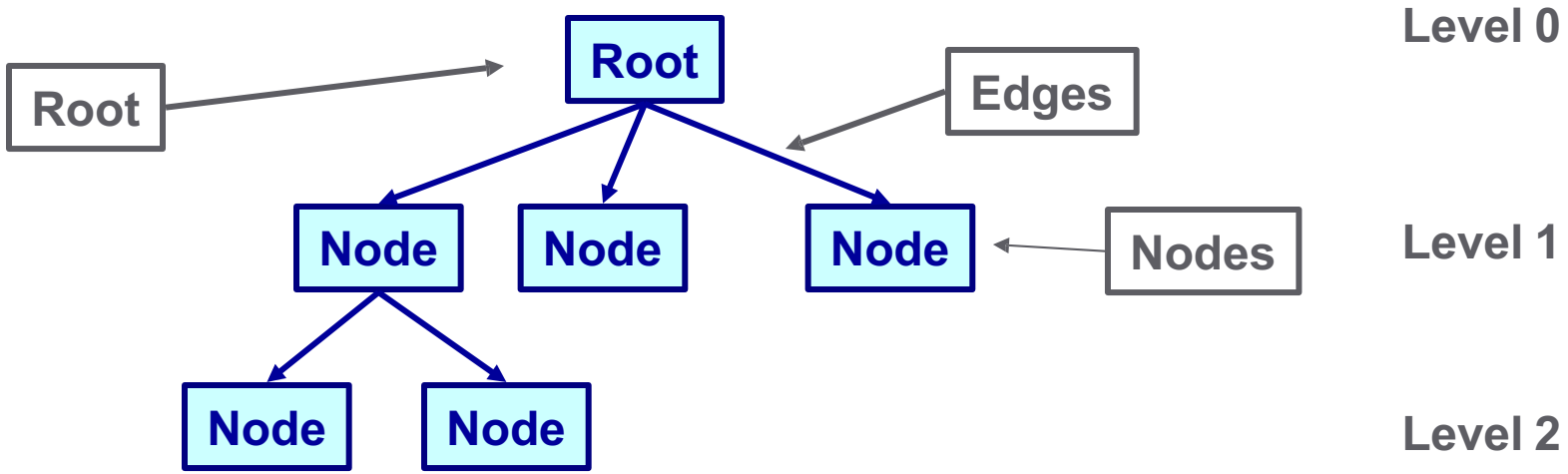


A *tree* is a non-linear structure in which elements are organized into a hierarchy

A tree is comprised of a set of *nodes* in which elements are stored and *edges* connect one node to another

Each node is located on a particular *level*

There is only one *root* node in the tree



# Trees

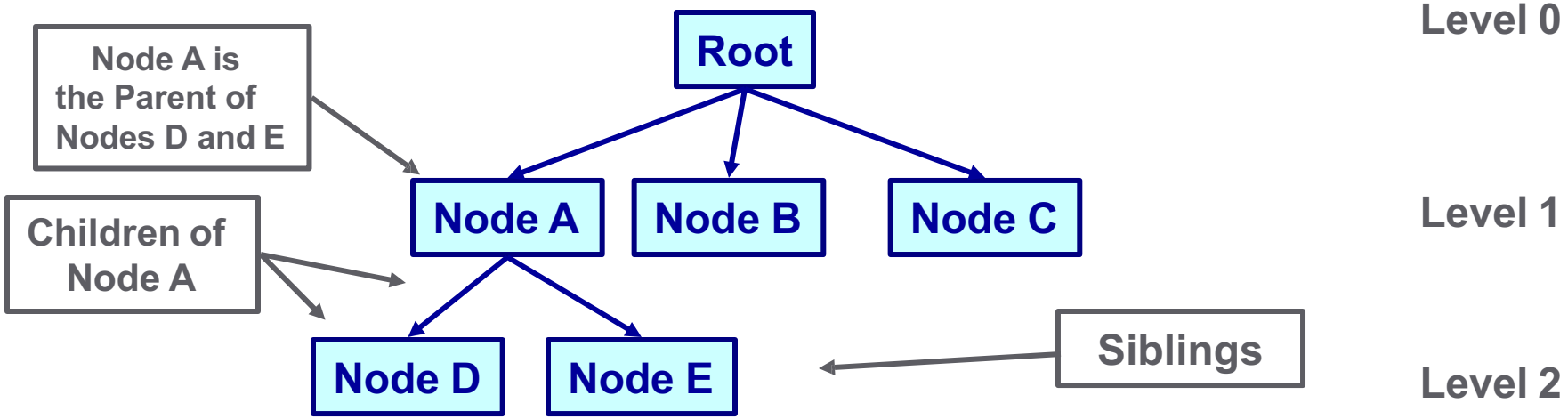


Nodes at the lower level of a tree are the *children* of nodes at the previous level

A node can have only one *parent*, but may have multiple children

Nodes that have the same parent are *siblings*

The root is the only node which has no parent



# Trees



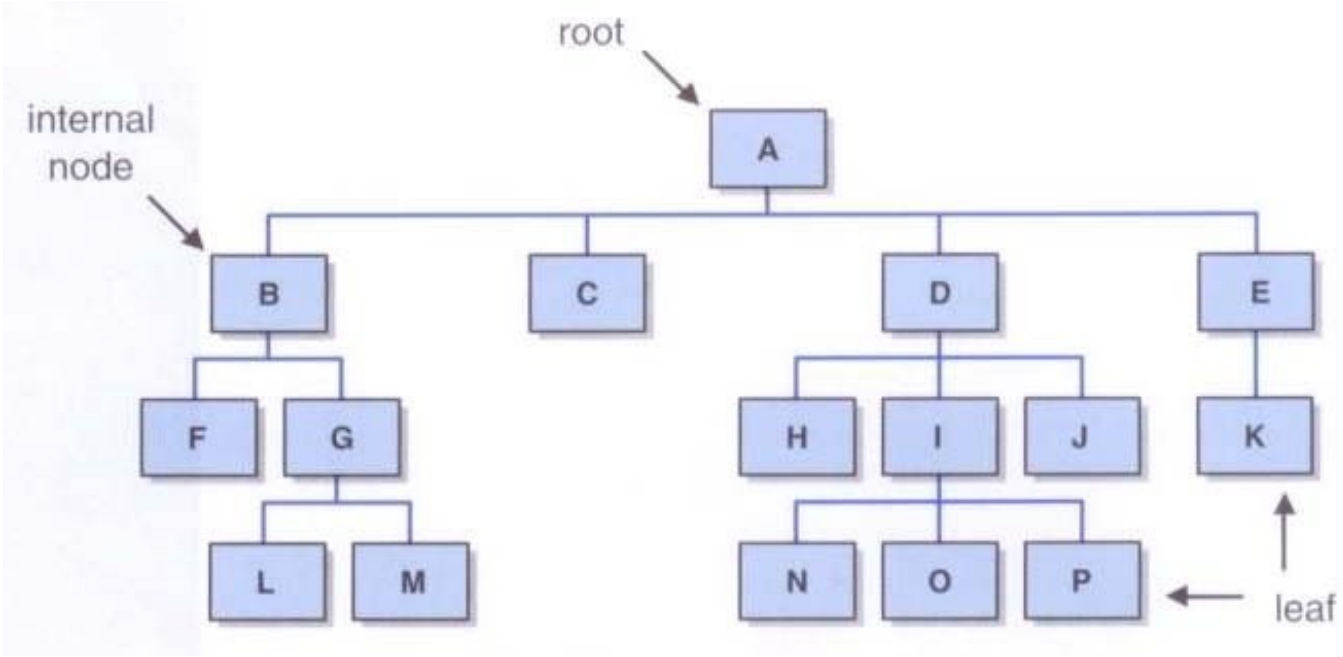
A node that has no children is a *leaf* node

A node that is not the root and has at least one child is an *internal node*

A *subtree* is a tree structure that makes up part of another tree

We can follow a *path* through a tree from parent to child, starting at the root

A node is an *ancestor* of a node if it is above it on the path from the root.



# Trees

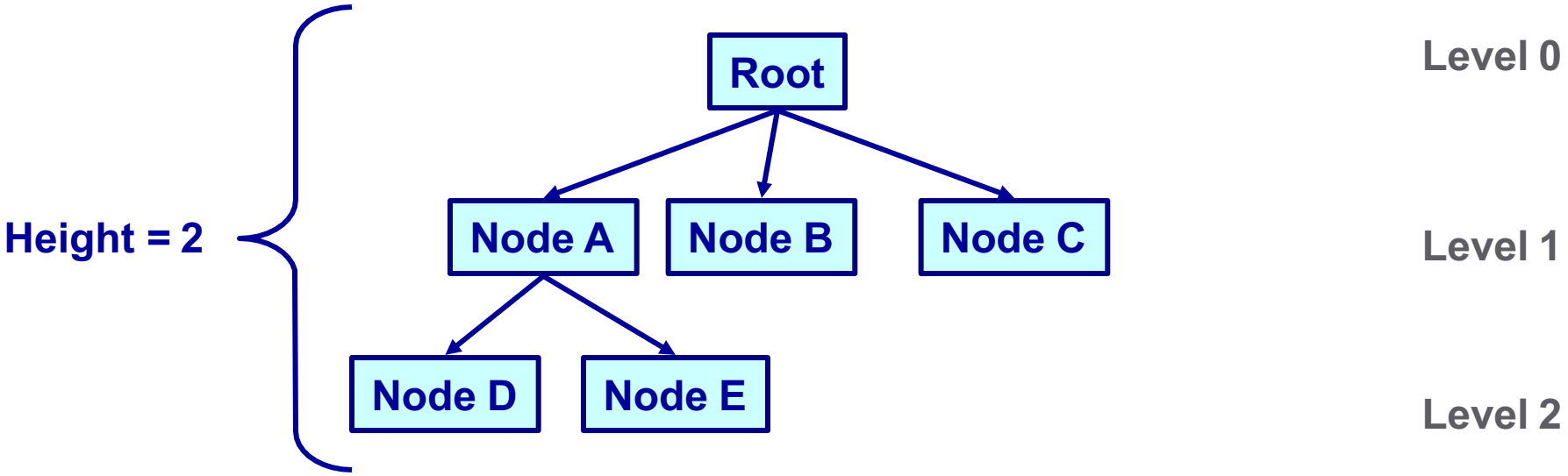


Nodes that can be reached by following a path from a particular node are the *descendants* of that node

The *level* of a node is the length of the path from the root to the node

The *path length* is the number of edges to get from the root to the node

The *height* of a tree is the length of the longest path from the root to a leaf



# Trees – Quiz

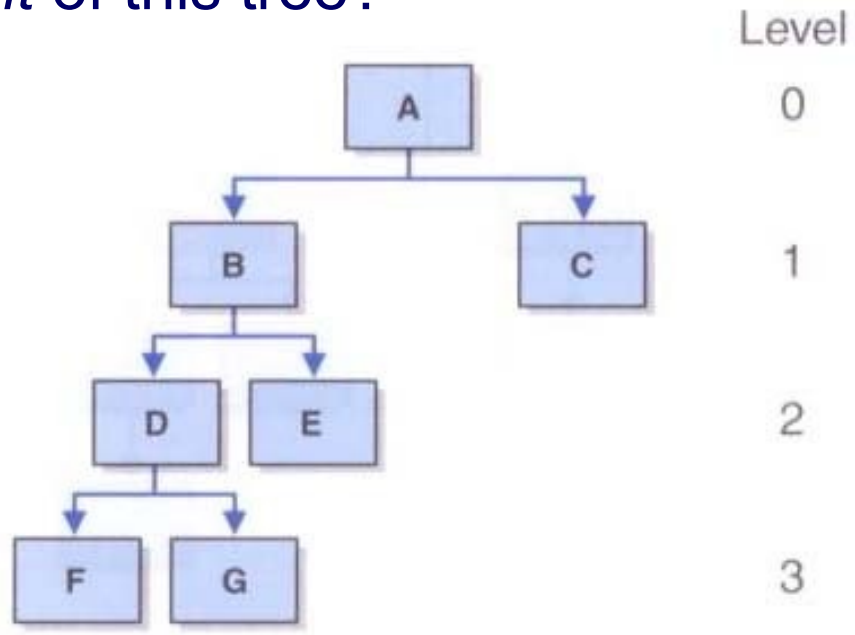


What are the *descendents* of node B?

What is the *level* of node E?

What is the *path length* to get from the root to node G?

What is the *height* of this tree?



# Classifying Trees



Trees can be classified in many ways

One important criterion is the maximum number of children any node in the tree may have

This may be referred to as the *order of the tree*

*General trees* have no limit to the number of children a node may have

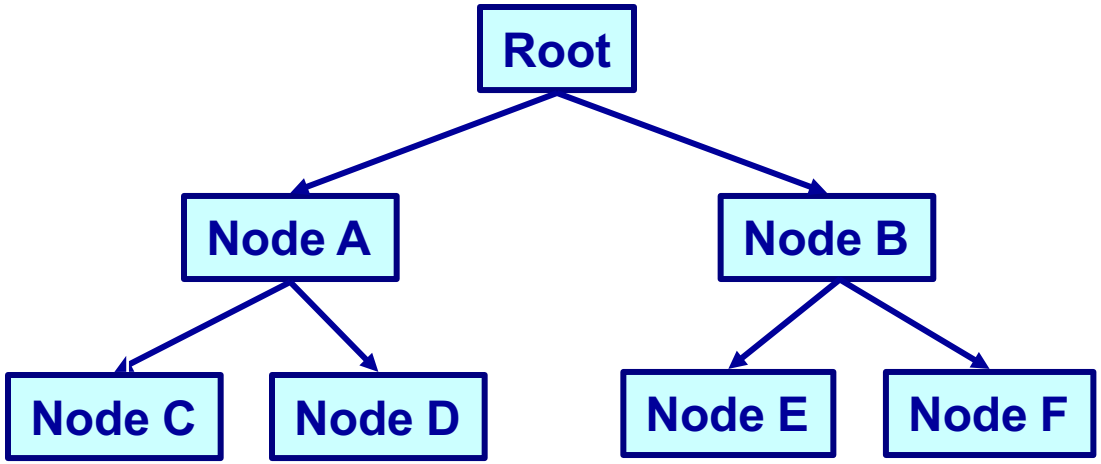
A tree that limits each node to no more than *n* children is referred to as an *n-ary* tree

The tree for play TicTacToe is an 9-ary tree (at most 9 moves)

# Binary Trees



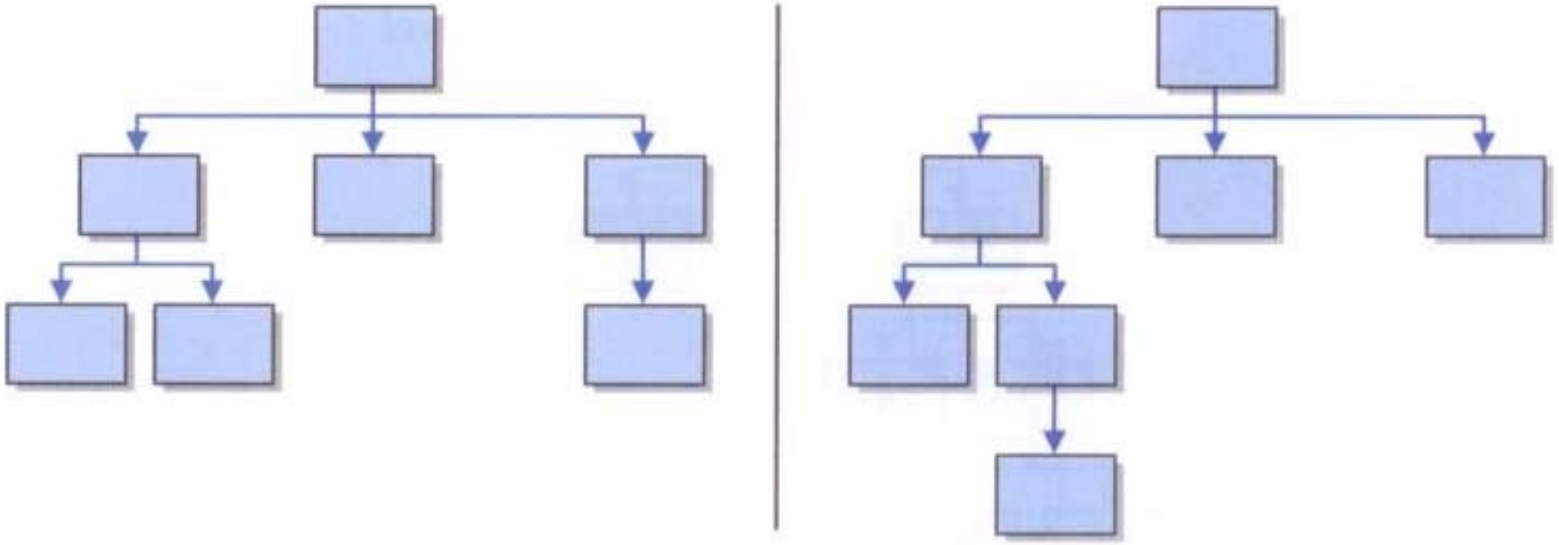
Trees in which nodes may have at most two children are called *binary trees*



# Balanced Trees



A tree is *balanced* if all of the leaves of the tree are on the same level or within one level of each other



balanced

unbalanced

# Full and Complete Trees

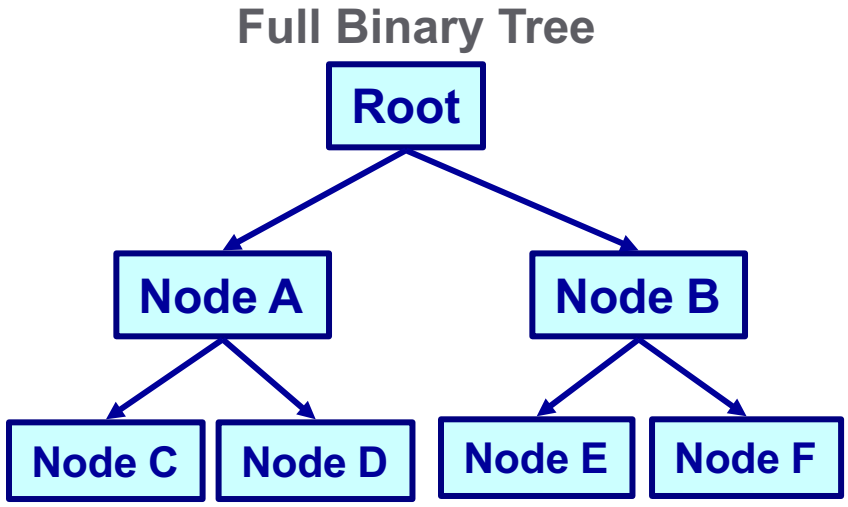


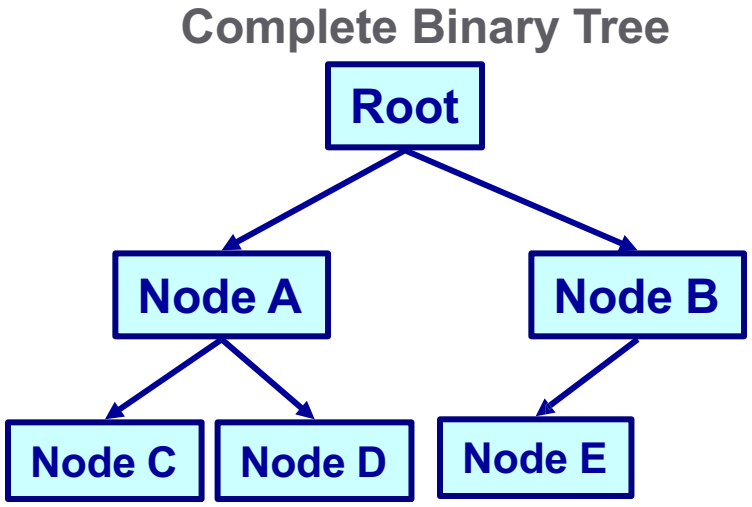
A balanced n-ary tree with m elements has a height of $\log_n m$

A balanced binary tree with n nodes has a height of $\log_2 n$

An n-ary tree is *full* if all leaves of the tree are at the same height and every non-leaf node has exactly n children

A tree is *complete* if it is full, or full to the next-to-last level with all leaves at the bottom level on the left side of the tree

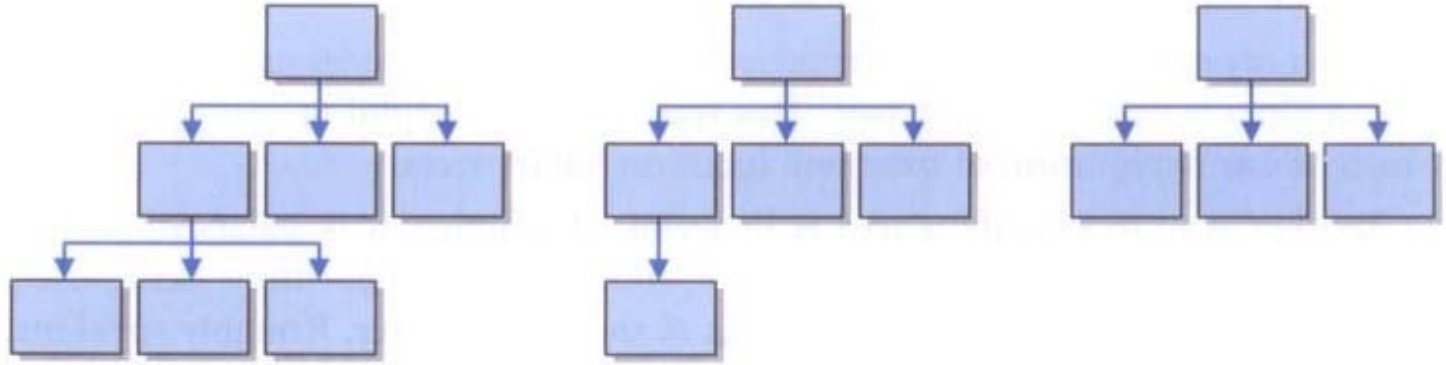


# Full and Complete Trees



Three complete trees:




Which trees are full?

Only tree c is full

# Implementing Trees



An obvious choice for implementing trees is a linked structure

Array-based implementations are the less obvious choice, but are sometimes useful

Let's first look at the strategy behind two array-based implementations

# Computed Child Links

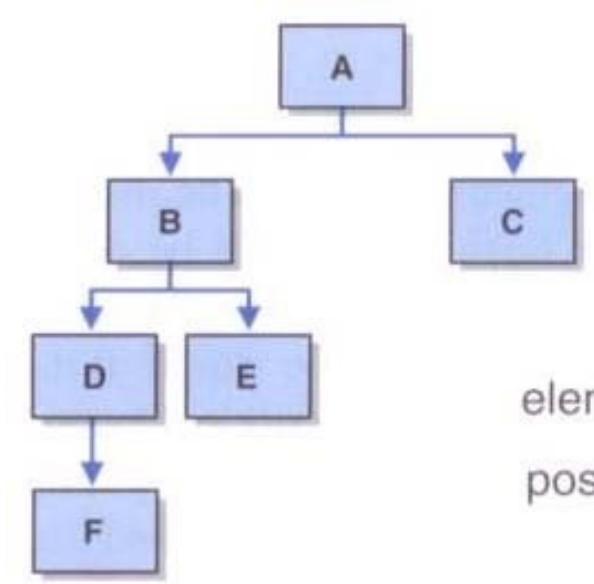


For full or complete trees, can use an array to represent a tree

For a binary tree with any element stored in position n,

- the element's left child is stored in array position (2n+1)
- the element's right child is stored in array position (2*(n+1))

If the represented tree is not complete or relatively complete, this approach can waste large amounts of array space




| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| element | A | B | C | D | E | | | F |
| position | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

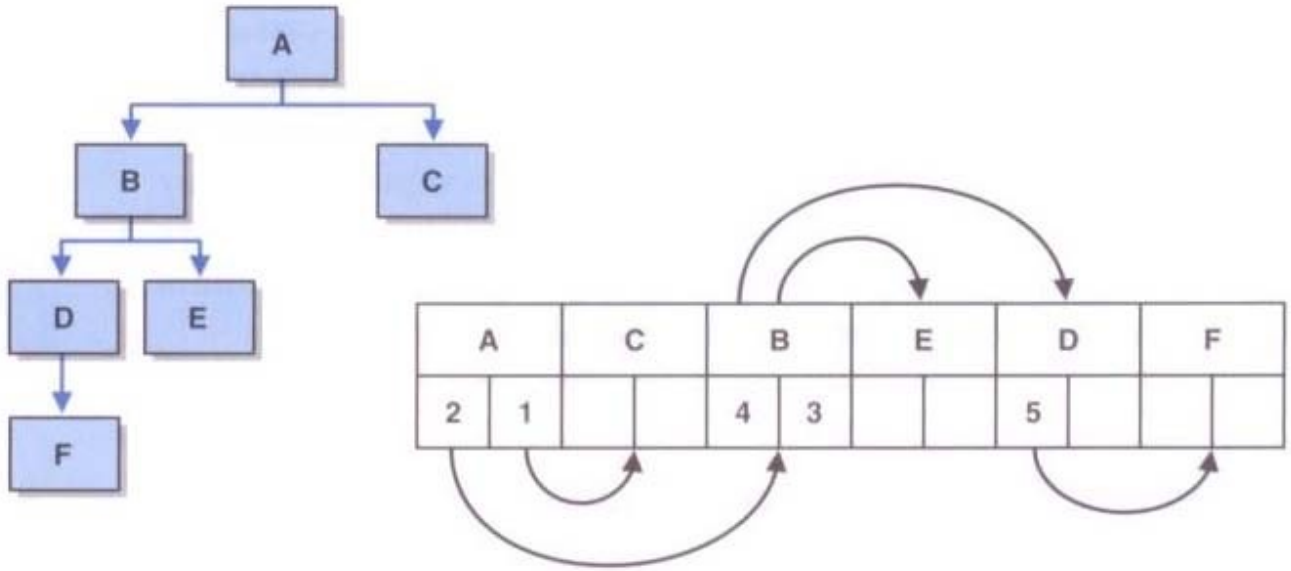
# Simulated Child Links



Each element of the array is an object that stores a reference to the tree element and the array index of each child

This approach is modeled after the way operating systems manage memory

Array positions are allocated on a first-come, first-served basis



# Tree Traversals



For linear structures, the process of iterating through the elements is fairly obvious (forwards or backwards)

For non-linear structures like a tree, the possibilities are more interesting

Let's look at four classic ways of *traversing* the nodes of a tree

All traversals start at the root of the tree

Each node can be thought of as the root of a subtree

# Tree Traversals



*Preorder*: visit the root, then traverse the subtrees from left to right

*Inorder*: traverse the left subtree, then visit the root, then traverse the right subtree

*Postorder*: traverse the subtrees from left to right, then visit the root

*Level-order*: visit each node at each level of the tree from top (root) to bottom and left to right

# Tree Traversals

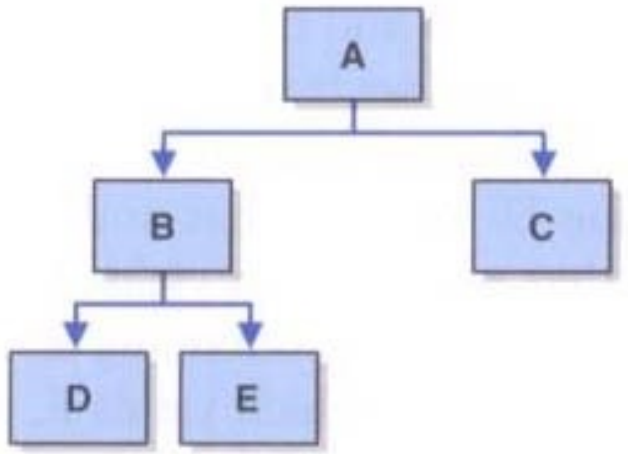


Preorder: A B D E C

Inorder: D B E A C

Postorder: D E B C A

Level-Order: A B C D E



# Tree Traversals



Recursion simplifies the implementation of tree traversals

Preorder (pseudocode):

```
Visit node
Traverse (left child)
Traverse (right child)
```

Inorder:

```
Traverse (left child)
Visit node
Traverse (right child)
```

# Tree Traversals



Postorder:

```
Traverse (left child)
Traverse (right child)
Visit node
```

A level-order traversal is more complicated

It requires the use of extra structures (such as queues and/or lists) to create the necessary order

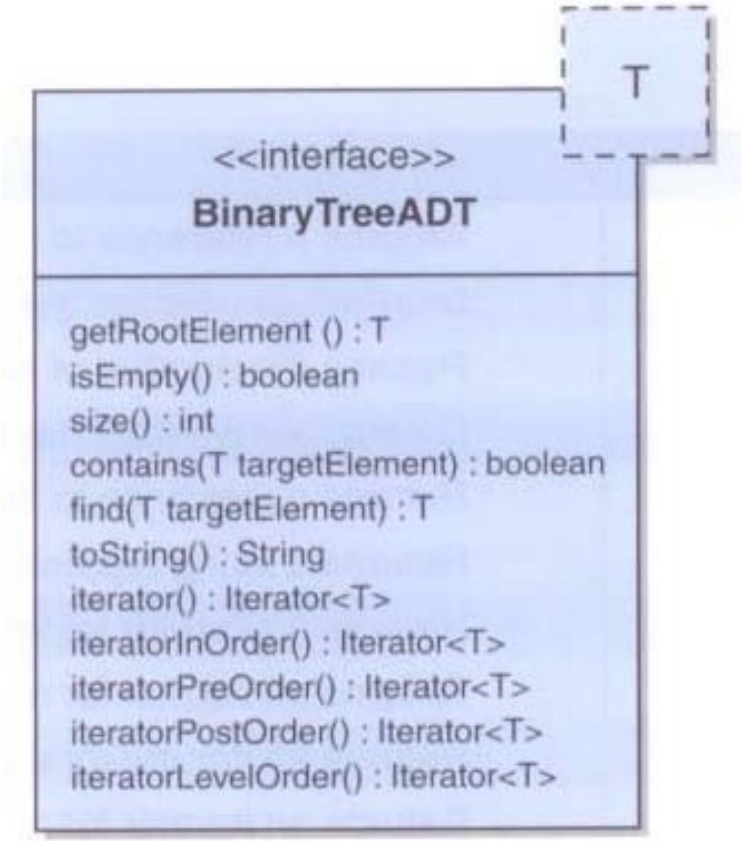
# A Binary Tree ADT



| Operation | Description |
|---|---|
| `getRoot` | Returns a reference to the root of the binary tree |
| `isEmpty` | Determines whether the tree is empty |
| `size` | Returns the number of elements in the tree |
| `contains` | Determines whether the specified target is in the tree |
| `find` | Returns a reference to the specified target element if it is found |
| `toString` | Returns a string representation of the tree |
| `iteratorInOrder` | Returns an iterator for an inorder traversal of the tree |
| `iteratorPreOrder` | Returns an iterator for a preorder traversal of the tree |
| `iteratorPostOrder` | Returns an iterator for a postorder traversal of the tree |
| `iteratorLevelOrder` | Returns an iterator for a level-order traversal of the tree |

# A Binary Tree ADT





# A Binary Tree ADT



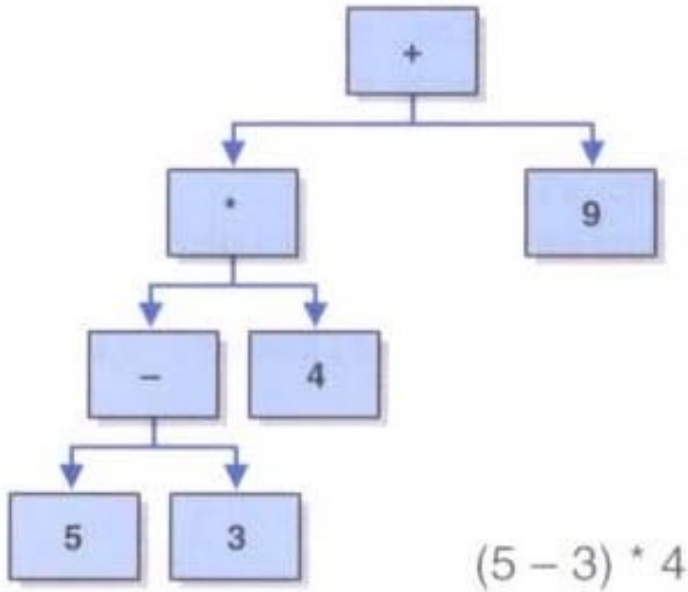
```
public interface BinaryTreeADT<T>
{
 public T getRootElement();
 public boolean isEmpty();
 public int size();
 public boolean contains(T targetElement);
 public T find(T targetElement);
 public String toString();
 public Iterator<T> iterator();
 public Iterator<T> iteratorInOrder();
 public Iterator<T> iteratorPreOrder();
 public Iterator<T> iteratorPostOrder();
 public Iterator<T> iteratorLevelOrder();
}
```

# Expression Trees



An *expression tree* is a tree that shows the relationships among operators and operands in an expression

An expression tree is evaluated from the bottom up



# Expression Tree Evaluation



Let's look at an example that creates an expression tree from a postfix expression and then evaluates it

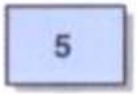
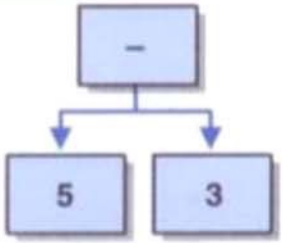
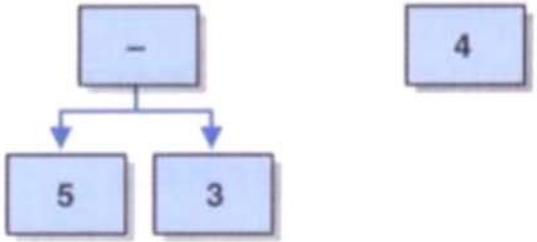
This is a modification of an earlier solution

It uses a stack of expression trees to build the complete expression tree

# Building an Expression Tree



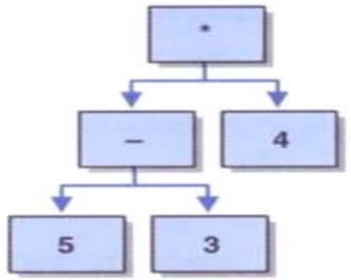
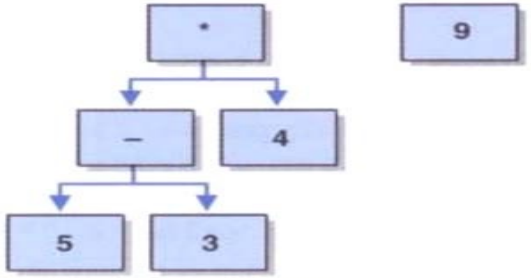
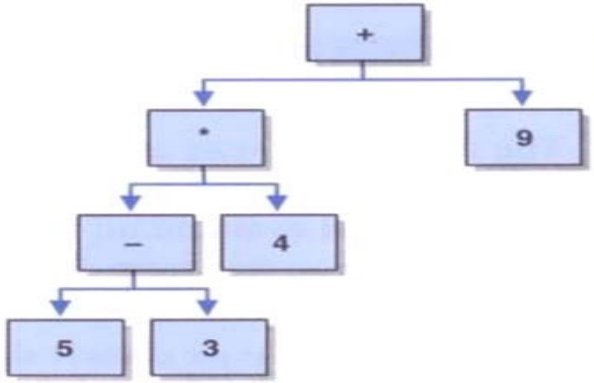
**Input in Postfix: 5 3 – 4 * 9 +**

| Token | Processing Steps | Expression Tree Stack (top at right) |
|---|---|---|
| 5 | push(new ExpressionTree(5, null, null) |   |
| 3 | push(new ExpressionTree(3, null, null) |   |
| – | op2 = pop<br>op1 = pop<br>push(new ExpressionTree(–, op1, op2) |   |
| 4 | push(new ExpressionTree(4, null, null) |   |

# Building an Expression Tree



**Input in Postfix: 5 3 – 4 * 9 +**

| | | |
|---|---|---|
| * | op2 = pop<br>op1 = pop<br>push(new ExpressionTree(*, op1, op2) |  |
| 9 | push(new ExpressionTree(9, null, null) |  |
| + | op2 = pop<br>op1 = pop<br>push(new ExpressionTree(+, op1, op2) |  |

# Decision Trees



A *decision tree* is a tree whose nodes represent decision points, and whose children represent the options available

The leaves of a decision tree represent the possible conclusions that might be drawn

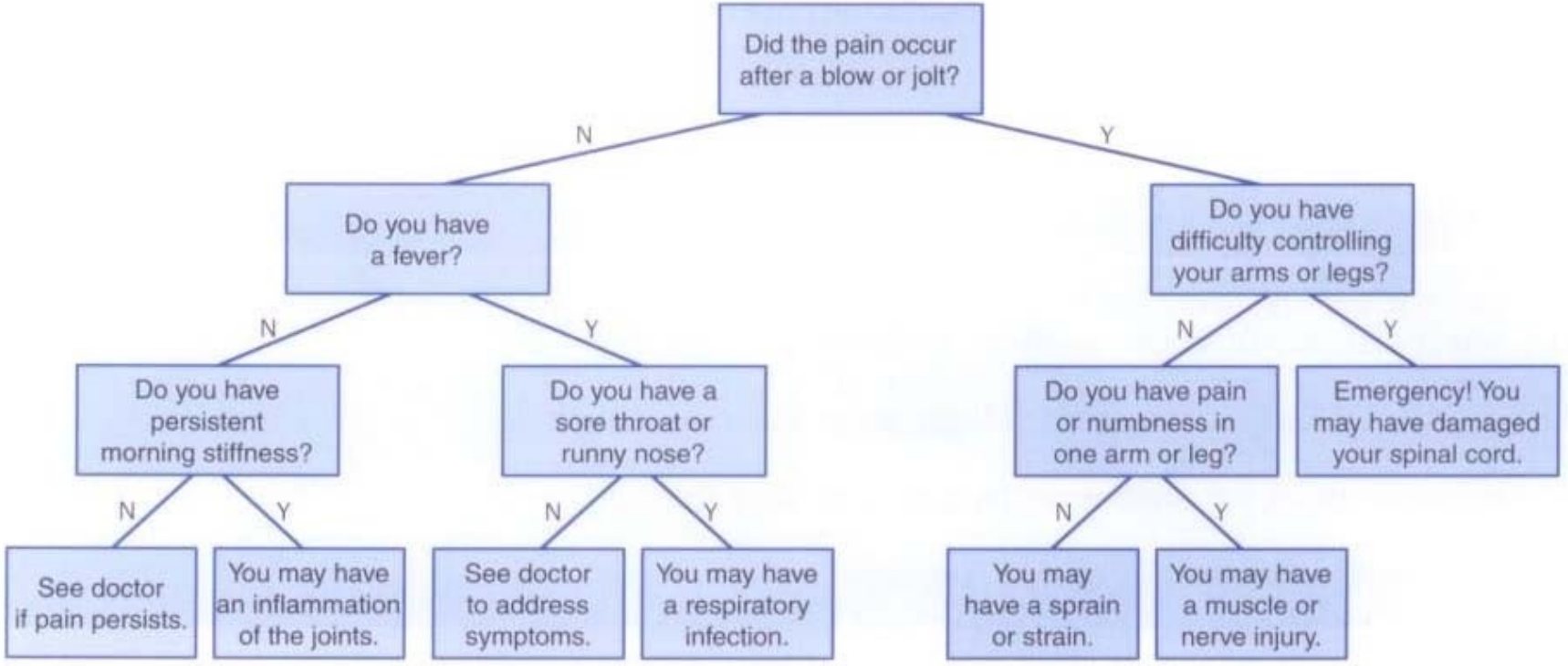
A simple decision tree, with yes/no questions, can be modeled by a binary tree

Decision trees are useful in diagnostic situations (medical, car repair, etc.)

# Decision Trees



A simplified decision tree for diagnosing back pain:



# Trees can be written to or read from a File



```
public DecisionTree(String filename) throws FileNotFoundException
{File inputFile  = new File(filename);
 Scanner scan    = new Scanner(inputFile);
 int numberNodes = scan.nextInt();
 scan.nextLine();
 int root = 0, left, right;

List<LinkedBinaryTree<String>> nodes = new java.util.ArrayList<LinkedBinaryTree<String>>();
  for (int i = 0; i < numberNodes; i++)
     nodes.add(i,new LinkedBinaryTree<String>(scan.nextLine()));

     while (scan.hasNext())
     {
      root  = scan.nextInt();
      left  = scan.nextInt();
      right = scan.nextInt();
      scan.nextLine();

      nodes.set(root, new LinkedBinaryTree<String>((nodes.get(root)).getRootElement(),
                                                   nodes.get(left), nodes.get(right)));
     }
     tree = nodes.get(root);
    }
```

# Implementing Binary Trees with Links



Now let's explore an implementation of a binary tree using links

The `LinkedBinaryTree` class holds a reference to the root

The `BinaryTreeNode` class represents each node, with links to a possible left and/or right child

# Implementing Binary Trees with Links



The `BinaryTreeNode` class represents each node, with links to a possible left and/or right child

```
public class BinaryTreeNode<T>
{
    protected T element;
    protected BinaryTreeNode<T> left, right;

    /**
     * Creates a new tree leaf node with the specified data.
     * @param obj the element that will become a part of the new tree node
    */
    public BinaryTreeNode(T obj)
    {
        element = obj;
        left    = null;
        right   = null;
    }
```

# Binary Trees with Links – Building Trees



The `BinaryTreeNode` constructor builds an internal or root node left and/or right child are supplied as parameters.

```
// Creates a new tree node with the specified left and right children
public BinaryTreeNode(T obj, LinkedBinaryTree<T> left, LinkedBinaryTree<T> right)
    {
        element = obj;
        if (left == null)
            this.left = null;
        else
            this.left = left.getRootNode();

         if (right == null)
            this.right = null;
        else
            this.right = right.getRootNode();
    }
```

# Binary Trees– Child Getters and Setters



```
public BinaryTreeNode<T> getRight()
    { return right; }

    public void setRight(BinaryTreeNode<T> node)
    { right = node; }

public BinaryTreeNode<T> getLeft()
    { return left;  }

public void setLeft(BinaryTreeNode<T> node)
    { left = node;  }
```

# Recall: Traversing a Maze Using Recursion



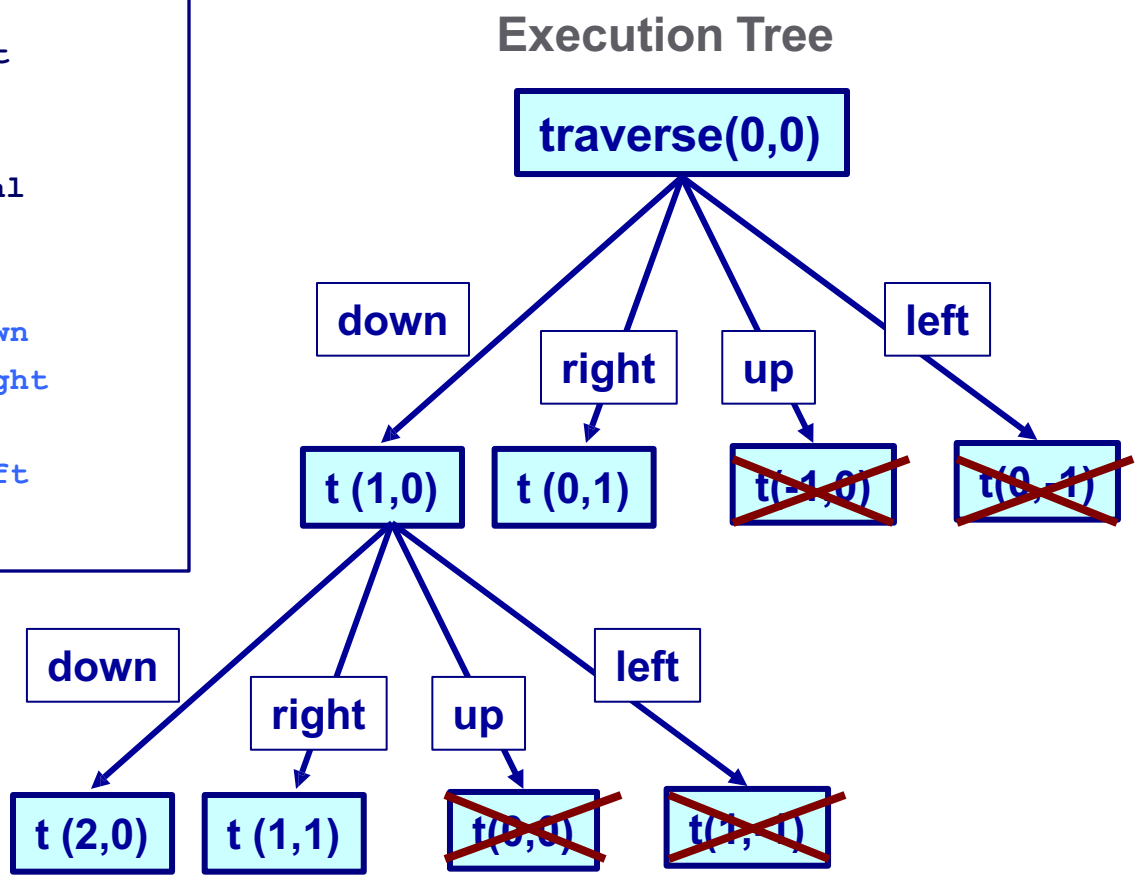
```
// Attempts to recursively traverse the maze.
public boolean traverse(int row, int column)
{  boolean done = false;
    if (maze.validPosition(row, column))
    {   maze.tryPosition(row, column);   // mark this cell as tried
        if (row == maze.getRows()-1 && column == maze.getColumns()-
        1)
            done = true;  // the maze is solved
        else
        {   done = traverse(row+1, column);       // down
            if (!done)
                done = traverse(row, column+1);  // right
            if (!done)
                done = traverse(row-1, column);  // up
            if (!done)
                done = traverse(row, column-1);  // left
        }
        if (done)  // this location is part of the final path
            maze.markPath(row, column);
    }
    return done;
}
}
```

# Execution Tree for understanding Program Flow



```
// Rrecursively traverse the maze.
public boolean traverse(int row, int
column)
{
  // Base Case – Test if reached goal
  else
  {
   traverse(row+1, column  ); // down
   traverse(row  , column+1); // right
   traverse(row-1, column  ); // up
   traverse(row  , column-1); // left
} }
```

```
9  13
1  1  1  0  1  1  0  0  0  1  1  1  1
1  0  0  1  1  0  1  1  1  1  0  0  1
1  1  1  1  1  0  1  0  1  0  1  0  0
0  0  0  0  1  1  1  0  1  0  1  1  1
1  1  1  0  1  1  1  0  1  0  1  1  1
1  0  1  0  0  0  0  1  1  1  0  0  1
1  0  1  1  1  1  1  1  0  1  1  1  1
1  0  0  0  0  0  0  0  0  0  0  0  0
1  1  1  1  1  1  1  1  1  1  1  1  1
```

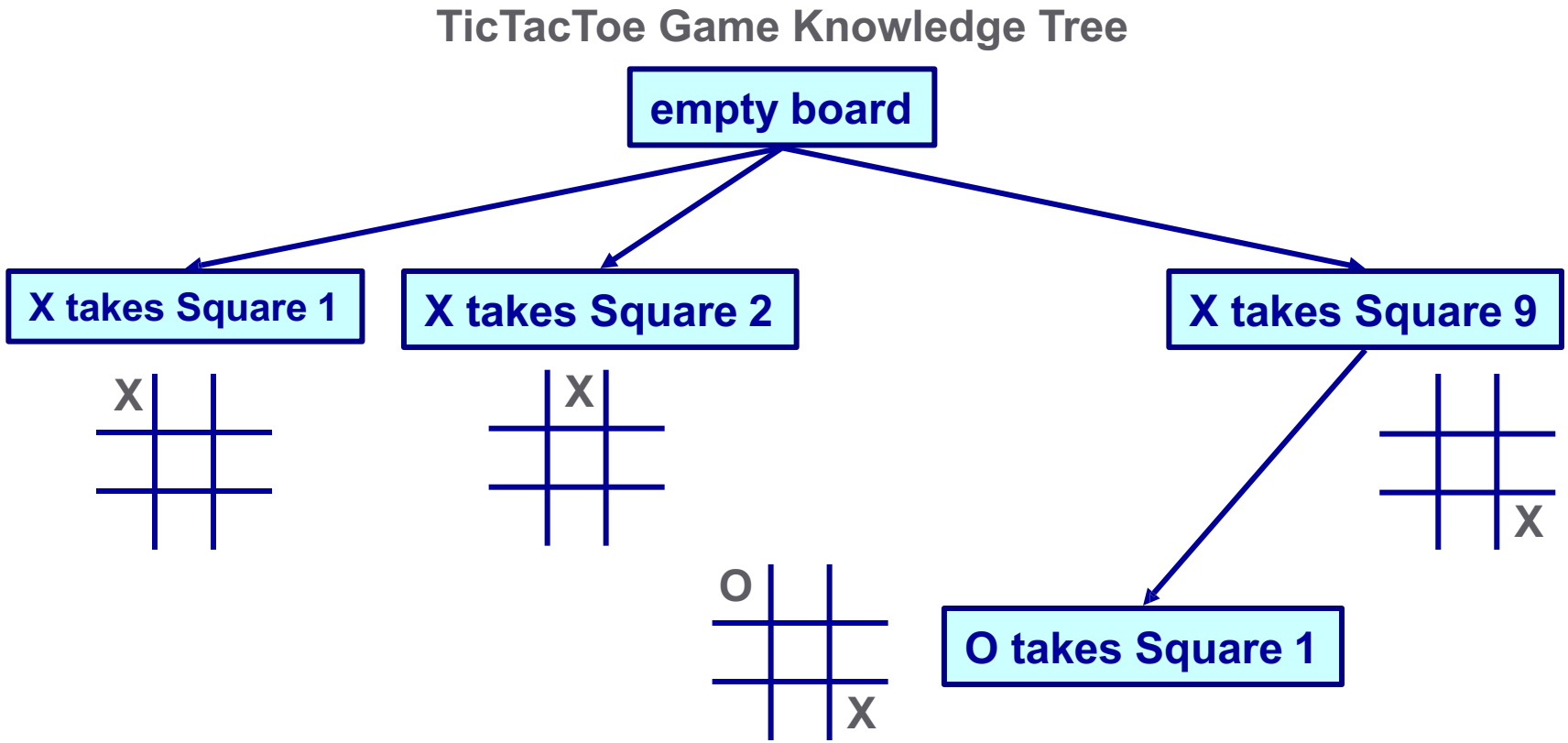


# Using Trees to Represent Knowledge - TicTacToe



As was seen with LearningPlayer in TicTacToe,

Trees can be used to represent and capture knowledge.



# Key Things to take away:



Trees:

- A tree is a nonlinear structure whose elements form a hierarchy
- Can be stored in an array using simulated link strategy
- Trees can be balanced or non-balanced
- A balanced N-ary tree with m elements has height $\log_n m$
- Four basic tree traversal methods: preorder, inorder, postorder, level order
- Preorder: visit the node first, then its children left to right
- Inorder: visit the left child, then the node, then its right child
- Postorder: visit the children left to right, then visit the node itself
- Level-Order: visit all nodes in a level left to right, starting with the root
- Decision Trees can be read from files and used to create an expert system
- Execution Trees can be used to Describe Recursive program flow
- Trees can be used to store and update knowledge for training AI programs

# Binary Search Trees



Binary search tree processing

Using BSTs to solve problems

BST implementations

Strategies for balancing BSTs

# Binary Search Trees



A *search tree* is a tree whose elements are organized to facilitate finding a particular element when needed

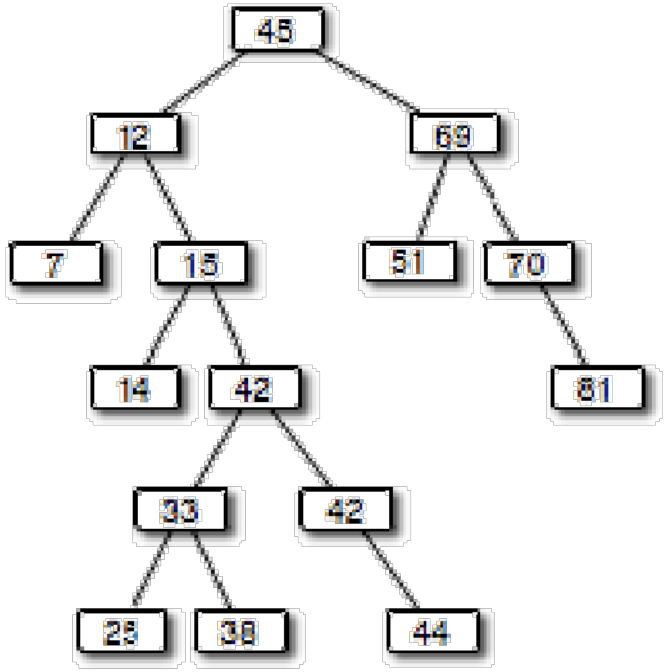
A *binary search tree* is a binary tree that, for each node *n*

- the left subtree of *n* contains elements less than the element stored in *n*
- the right subtree of *n* contains elements greater than or equal to the element stored in *n*

**This is *how***

Google

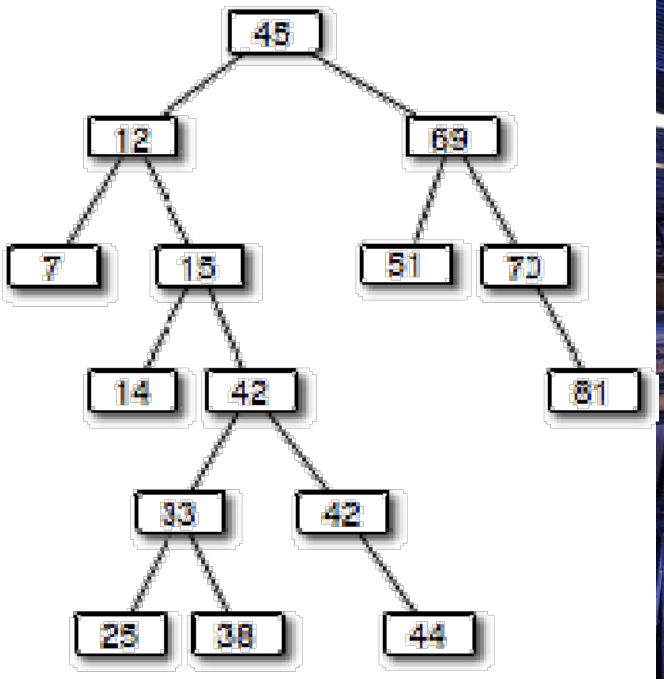
**finds your websites!**

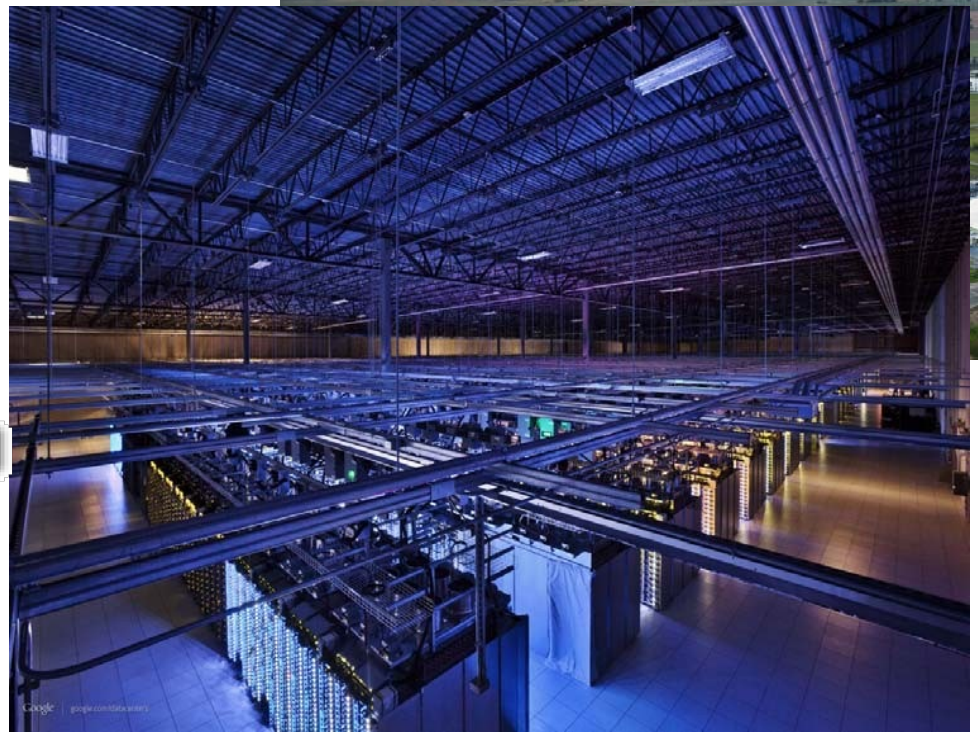


# How Google finds Websites



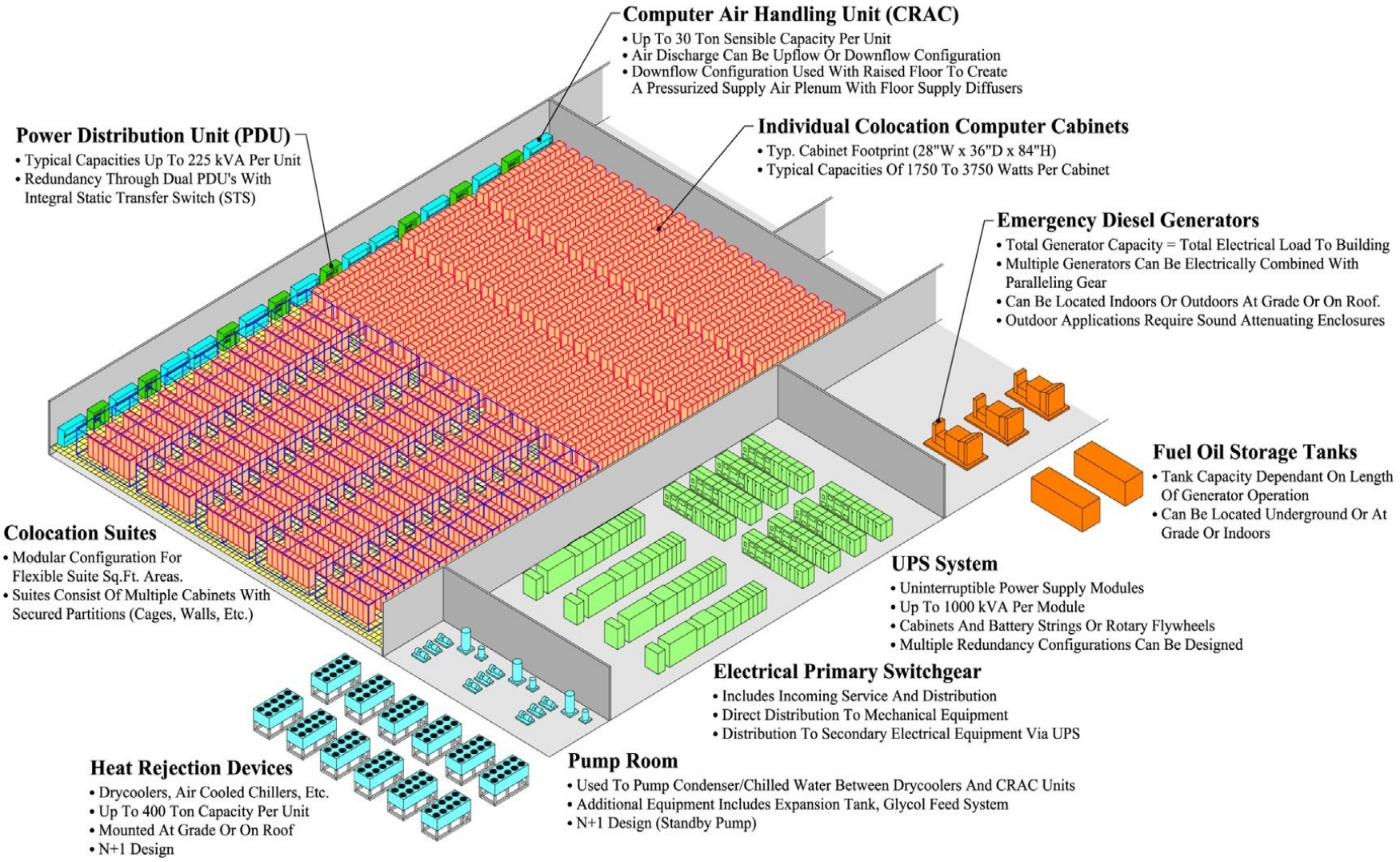
Web Crawlers search the Internet for new websites
Read every webpage and every word
HUGE files of data – Petabytes!
Data Centers process this data
And Update Google Search Trees
Every webpage, every day



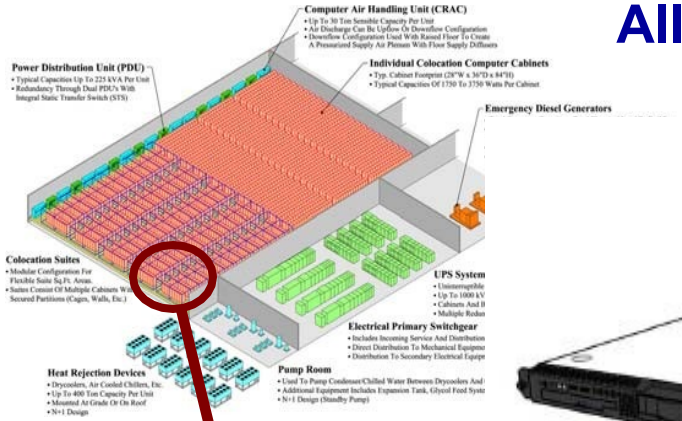
# Google and Amazon Data Centers





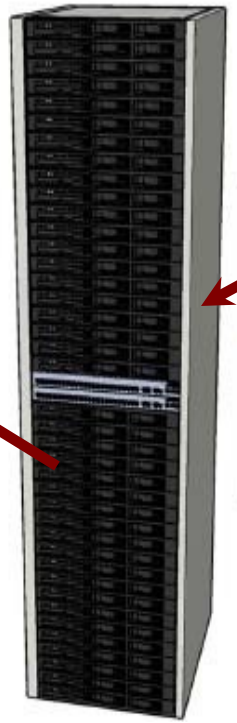
# Data Centers – A Closer Look

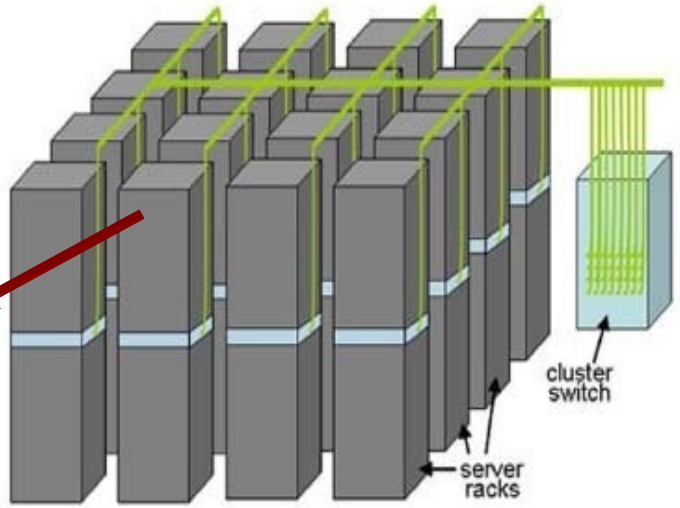


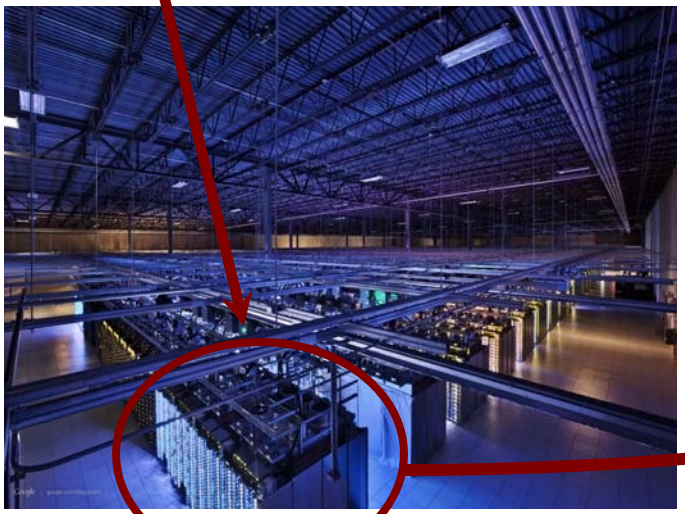



**All this just to update some search trees**

**Some VERY BIG search trees!**





**For more info:**
**Dr. Mohamed Hefeeda**
**Big-Data and Multimedia**

# Binary Search Trees



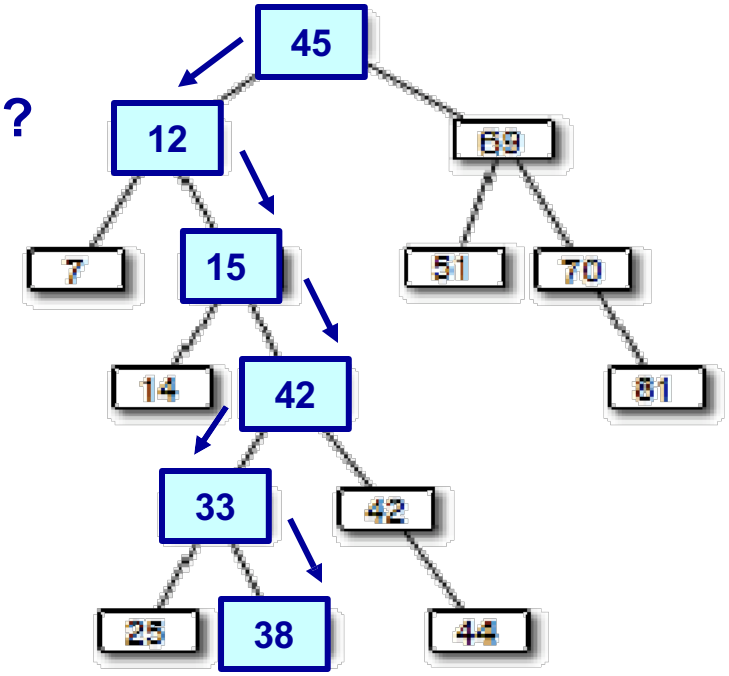
To determine if a particular value exists in a tree

- start at the root
- compare target to element at current node
- move left from current node if target is less than element in the current node
- move right from current node if target is greater than element in the current node

We eventually find the target or hit the end of a path (target is not found)

**How to find node with *key* value 38?**

- **Start at Root and compare 38 to 45**
- **38 < 45 so go to left subtree**
- **38 > 12 so go to the right**
- **38 > 15 so go to the right again**
- **38 < 42 so go left**
- **38 > 33 so go right**
- **38 found!**
- **Return Object stored at this node**



# Binary Search Trees



- The particular shape of a binary search tree depends on the order in which the elements are added
- The shape may also be dependant on any additional processing performed on the tree to reshape it
- Binary search trees can hold any type of data, so long as we have a way to determine relative ordering
- Objects implementing the `Comparable` interface provide such capability

# Binary Search Trees

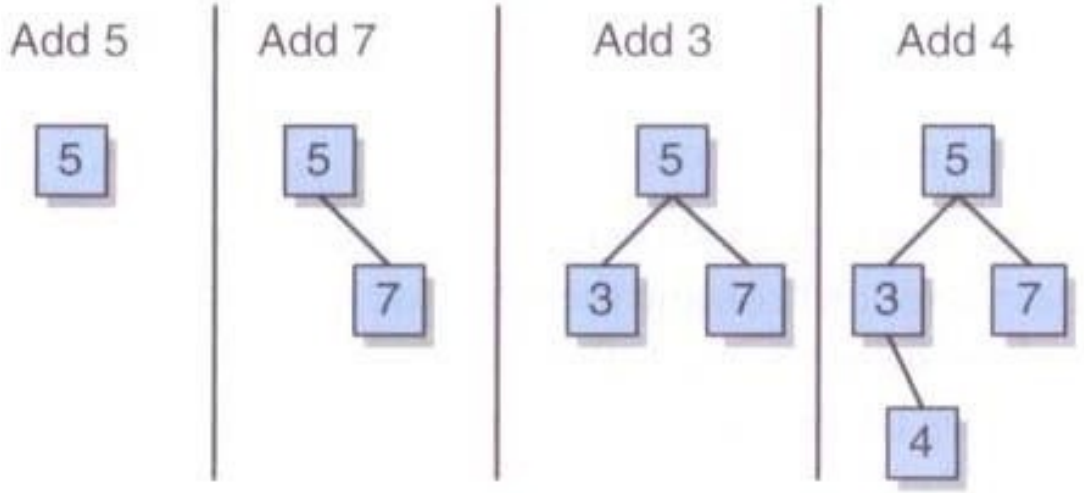


Process of adding an element is similar to finding an element

New elements are added as leaf nodes

Start at the root, follow path dictated by existing elements until you find no child in the desired direction

Then add the new element



# Binary Search Trees



BST operations:

| Operation | Description |
|---|---|
| `addElement` | Add an element to the tree. |
| `removeElement` | Remove an element from the tree. |
| `removeAllOccurrences` | Remove all occurrences of element from the tree. |
| `removeMin` | Remove the minimum element in the tree. |
| `removeMax` | Remove the maximum element in the tree. |
| `findMin` | Returns a reference to the minimum element in the tree. |
| `findMax` | Returns a reference to the maximum element in the tree. |

# BST Element Removal

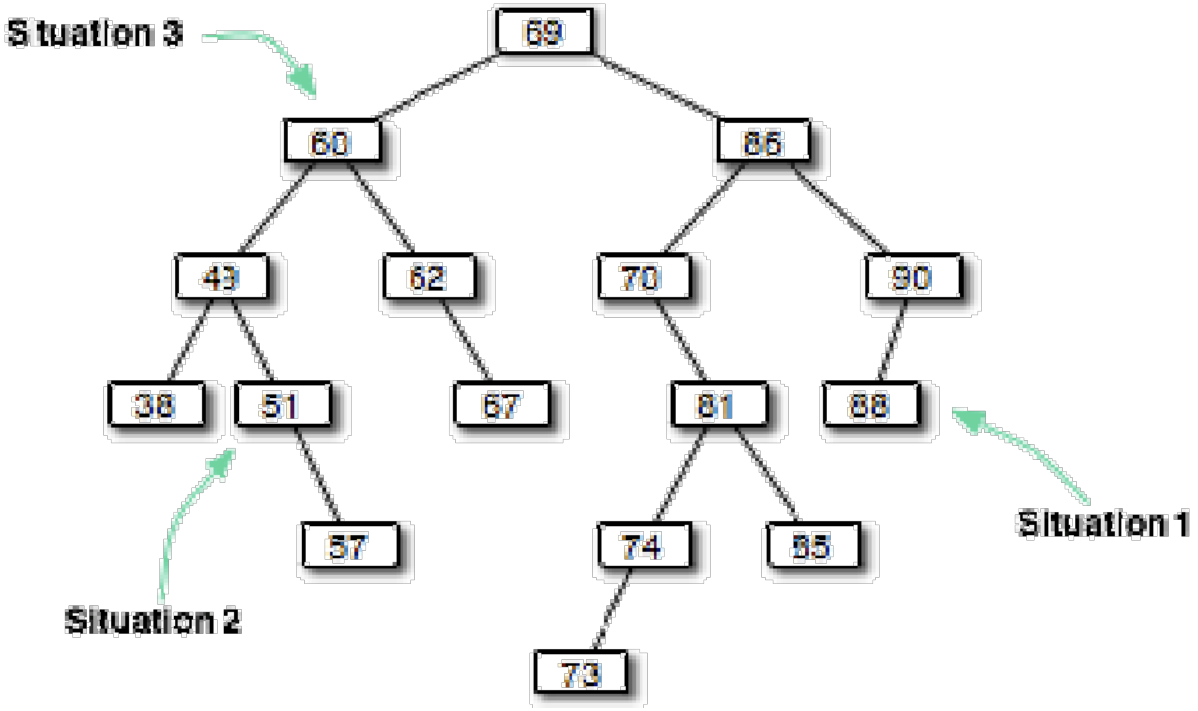


Removing a target in a BST is not as simple as that for linear data structures

After removing the element, the resulting tree must still be valid

Three distinct situations must be considered when removing an element

1. Node to remove is a leaf
2. Node to remove has one child
3. Node to remove has two children

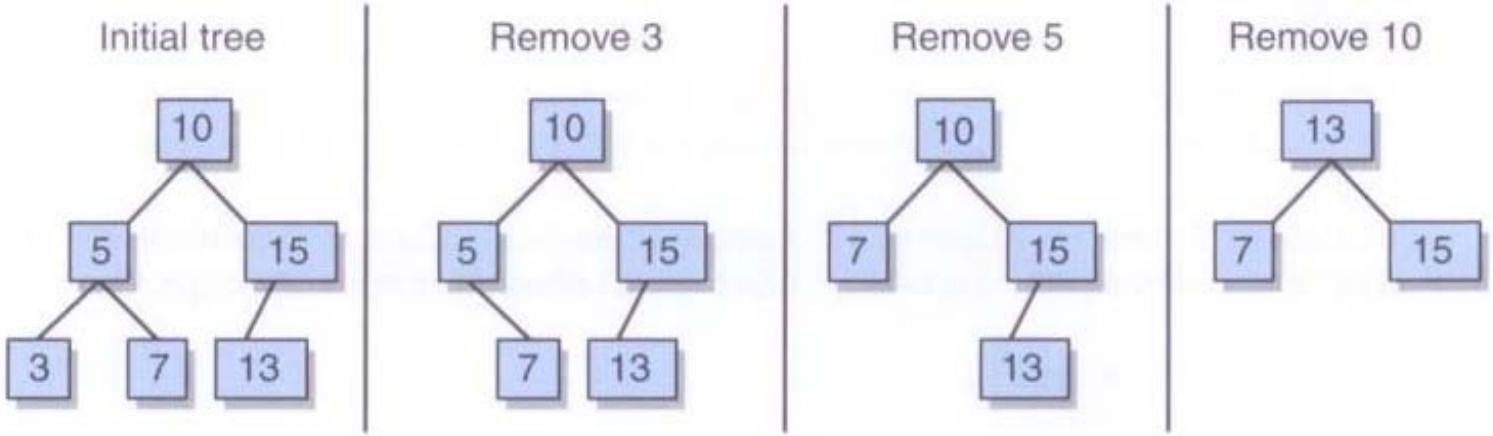


# BST Element Removal



## Dealing with the situations

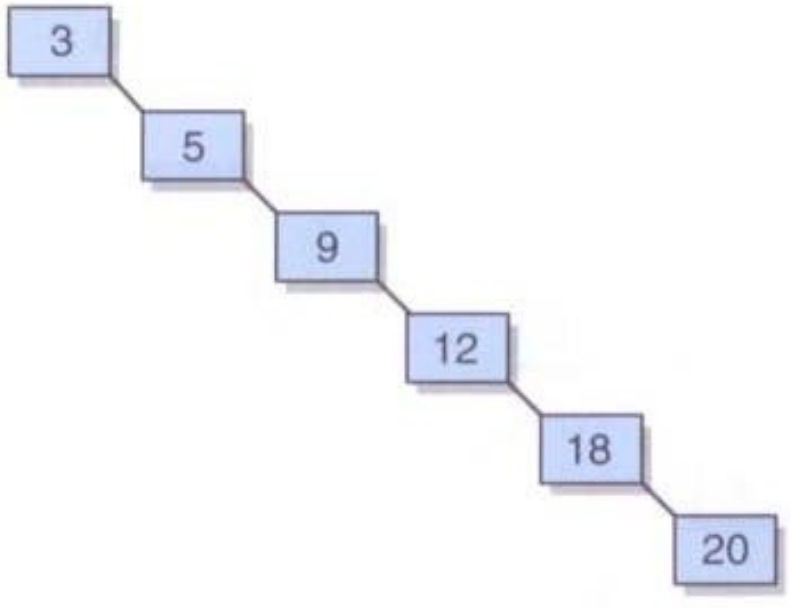
- Node is a leaf: it can simply be deleted
- Node has one child: the deleted node is replaced by the child
- Node has two children: an appropriate node is found lower in the tree and used to replace the node
  - Good choice: *inorder successor* (node that follows in an inorder traversal)
  - The inorder successor is guaranteed not to have a left child
  - Thus, removing the inorder successor to replace the deleted node will result in one of the first two situations (it's a leaf or has one child)



# Balancing BSTs



As operations are performed on a BST, it could become highly unbalanced (a *degenerate tree*)



# Balancing BSTs



Text implementation does not ensure the BST stays balanced

Other approaches do, such as AVL trees and red/black trees

We will explore *rotations* – operations on binary search trees to assist in the process of keeping a tree balanced

Rotations do not solve all problems created by unbalanced trees, but show the basic algorithmic processes that are used to manipulate trees

# Balancing BSTs

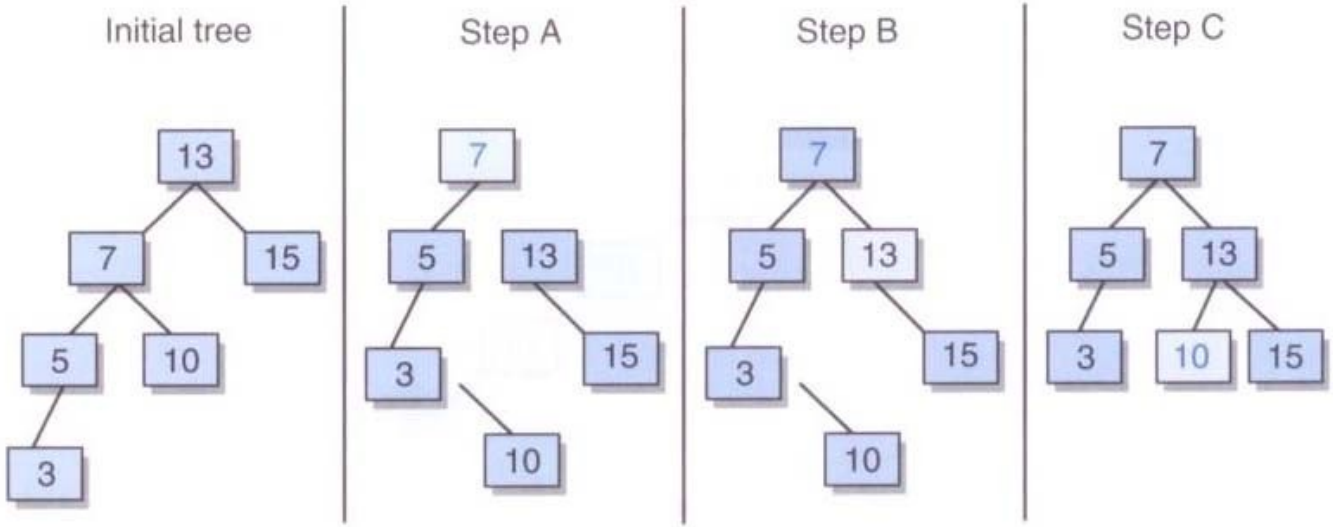


A *right rotation* can be performed at any level of a tree, around the root of any subtree

Corrects an imbalance caused by a long path in the left subtree of the left child of the root

To correct the imbalance

- A: Make the left child element of the root the new root element
- B: Make the former root element the right child element of the new root
- C: Make the right child of what was the left child of the former root



# Balancing BSTs



A *left rotation* can be performed at any level of a tree, around the root of any subtree

Corrects an imbalance caused by a long path in the right subtree of the left child of the root

To correct the imbalance

- A: Make the right child element of the root the new root element
- B: Make the former root element the left child element of the new root
- C: Make the left child of what was the right child of the former root, the new right child of the former root